

10-Python Strings

Assignment Instructions

- Assignments must be submitted by uploading them to a shared folder, with each student creating a folder named after themselves.
- Submission deadline is March 10th; late assignments are not accepted. All assignments are graded.

Introduction to Strings

- Python strings are fundamental data types and represent any content enclosed in single, double, or triple quotes.
- Identifiers are names without quotes, while anything inside quotes is a string object.

Memory Management for Strings

- Strings and integers utilize object reusability (caching) in certain ranges (integers: -5 to 256; specific characters for strings are pre-allocated).
- Strings are stored at both the string and character level in memory (heap and variable area distinction explained).
- String interning involves pre-allocating memory for common characters (A-Z, a-z, 0-9, _). Repeated characters share memory via references.

String Indexing and Slicing

Indexing

- Accessing string characters via zero-based positive indexing (from the left) and negative indexing (from the right, starting at -1).
- Negative indexing and positive indexing both allow efficient character access, with no strict rule for when to use which.
- Examples provided for accessing first and last characters and with both index types.

Slicing

- Allows retrieval of sub-strings using the format [start_index:end_index:step].
- End index is always exclusive, and step can be positive (left-to-right) or negative (right-to-left), with default step of +1.
- Use-cases include reversing strings, extracting specific ranges, and skipping characters (with steps >1).
- Examples provided for dynamic slicing, default values, and use in real-world contexts (e.g., extracting usernames).

Properties and Operations on Strings

- Strings are immutable: item assignment is not allowed; all string transformations return new string objects rather than modifying originals.
- Strings are sequences: ordered collections supporting positive/negative indexing, concatenation (using '+'), repetition (using '*', integer only), and membership checks (using 'in' and 'not in').
- Concatenation and repetition always produce new objects, not altering the originals.

- Membership operations are case sensitive and require correct order; working for single or multiple characters.

String Methods and Type Checking

- Case-changing methods:

- `.capitalize()`: Capitalizes only the first letter.
- `.title()`: Capitalizes the first letter of each word.
- `.upper()`, `.lower()`, `.swapcase()`: Change case of all letters; swapcase inverts case.

- Type-checking methods (all return boolean):

- `.isalnum()`, `.isalpha()`, `.isdecimal()`, `.isdigit()`, `.isnumeric()`

: Checks for alphanumeric, alphabetic, decimal, digit, general numeric values; explanations given for subtle differences (e.g., superscript/subscript handling).

- `.isascii()`: True if all characters are ASCII; false otherwise.

- `.isidentifier()`: Checks Python identifier validity (except for keywords, which require the `keyword` module for full verification).

- `.islower()`, `.isupper()`, `.istitle()`, `.isspace()`, `.isprintable()`

: For checking lowercase, uppercase, titlecase, all-spaces, or printable-character status.

- Detailed examples are provided for each, including special notes on unicode numerals, ASCII exceptions, and specific use-cases (e.g., why keywords like 'for' need a different check).

Assignment and Evaluation Details

- Next assignment on strings and loops is due on March 17th; all assignments are graded and will be updated accordingly.
- Weekend extension for assignment submission was requested.

Summary

This session provided an in-depth examination of Python strings, including definitions, memory management, indexing/slicing, properties, basic operations, and a suite of built-in methods. The session focused on hands-on practical usage, illustrated with examples, and set the framework for further exploration in upcoming lectures. Assignment logistics and grading procedures were also clarified.

Notes powered by [CraftNote](#)